8. Functions

Task 1:

Code:

```
import Foundation
func average(of marks: Int...) -> Double {
 if marks.isEmpty {
 return 0 }
 let total = marks.reduce(0, +)
 return Double(total) / Double(marks.count) }
func grade(for average: Double) -> String {
 switch average {
 case 90...:
 return "O"
 case 80..<90:
 return "A+"
 case 70..<80:
 return "A"
 case 60..<70:
 return "B"
 default:
 return "RA"
 } }
func stats(of marks: [Int]) -> (min: Int, max: Int, average: Double)? {
 guard !marks.isEmpty else {
 return nil
 }
 let minimum = marks.min()!
 let maximum = marks.max()!
```

```
 let avg = Double(marks.reduce(0, +)) / Double(marks.count)
 return (min: minimum, max: maximum, average: avg)
}
func curve(_ marks: inout [Int], by bonus: Int = 5) {
 for index in marks.indices {
 marks[index] = min(100, marks[index] + bonus)
 } }
print("Average:", average(of: 80, 90, 70))
print("Empty average:", average())
print("Grade:", grade(for: 85))
let marks = [70, 85, 95]
if let result = stats(of: marks) {
 print("Minimum:", result.min)
 print("Maximum:", result.max)
 print("Average:", result.average)
}
var studentMarks = [70, 95, 100]
curve(&studentMarks)
print("After bonus:", studentMarks)
curve(&studentMarks, by: 10)
print("After extra bonus:", studentMarks)
```

OUTPUT:

Output

```
Average: 80.0
Empty average: 0.0
Grade: A+
Minimum: 70
Maximum: 95
Average: 83.33333333333333
After bonus: [75, 100, 100]
After extra bonus: [85, 100, 100]
```

Task 2:

Code:

```
import Foundation
func isPalindrome(_ text: String) -> Bool {
 let cleaned = text.lowercased().filter { $0.isLetter || $0.isNumber }
 return cleaned == String(cleaned.reversed())
}
func vowelCount(in text: String) -> Int {
 let vowels = "aeiou"
 return text.lowercased().filter { vowels.contains($0) }.count
}
func wordCount(in text: String) -> Int {
 text.split { $0.isWhitespace }.count
}
func wordCount(in text: String, minLength: Int) -> Int {
 guard minLength > 0 else { return 0 }
 return text.split { $0.isWhitespace }
 .filter { word in
 word.filter { $0.isLetter }.count >= minLength
 }
 .count
}
let sentence = "A man, a plan, a canal: Panama"
print("Palindrome:", isPalindrome(sentence))
print("Vowels:", vowelCount(in: sentence))
print("Words:", wordCount(in: sentence))
print("Words with at least 4 letters:",
 wordCount(in: sentence, minLength: 4))
```

OUTPUT:

```
Output

Palindrome: true
Vowels: 10
Words: 7
Words with at least 4 letters: 3
```

Task 3:

Correct Code:

```
import Foundation

func makeMultiplier(_ factor: Int) -> (Int) -> Int {
 return { number in
 number * factor
 }
}

let double = makeMultiplier(2)

let triple = makeMultiplier(3)

func apply(
 _ values: [Int],
 using transform: (Int) -> Int
) -> [Int] {
 return values.map(transform)
}

let numbers = [1, 2, 3, 4, 5]

print("Doubled:", apply(numbers, using: double))

print("Tripled:", apply(numbers, using: triple))

print("Squared:", apply(numbers, using: { number in
 number * number
```

}))

OUTPUT:

```
Output

Doubled: [2, 4, 6, 8, 10]
Tripled: [3, 6, 9, 12, 15]
Squared: [1, 4, 9, 16, 25]
```

Functions are first-class values

In Swift, functions can be stored in variables, passed as arguments to other functions, and returned from functions.

In this program, double and triple store functions returned by makeMultiplier(). These functions are then passed to apply() as arguments. This demonstrates that functions are first-class values in Swift.